Starter Xanadu PennyLane infrastructure pack

Here's a **starter PennyLane "infrastructure pack"** you can paste into a repo. It's designed to:

1. define the **community-wide sensor placement QUBO** (maximize avoided loss under budget),
2. run **QAOA in PennyLane** on qubit simulators/backends today (recommended), and
3. optionally support a **photonic simulation track** — with an important caveat: the PennyLane–Strawberry Fields plugin is **only compatible up to PennyLane 0.29** and is not supported on newer PennyLane versions.

So the plan is:

- **Track A (recommended for deployment now):** PennyLane 0.44 + QAOA on `default.qubit` / `lightning.qubit` and later on any gate-based backend reachable via plugins. PennyLane latest is 0.44.0.
- **Track B (photonic R&D):** a separate legacy environment (`pennylane==0.29` + `pennylane-sf`) or use Strawberry Fields directly.

0) Repo layout (copy/paste)

```
manzanillo-qc/
  README.md
  requirements.txt
  requirements-photonic-legacy.txt
  examples/
    config_small.yaml
  src/
    manzanillo_qc/
      __init__.py
      config.py
      instance.py
      qubo.py
      ising.py
      qaoa.py
      backends.py
      cli.py
```

1) `requirements.txt` (modern PennyLane, recommended)

```
pennylane==0.44.0
pennylane-lightning==0.44.0
numpy>=1.26
scipy>=1.11
pyyaml>=6.0
pydantic>=2.6
typer>=0.12
rich>=13.7
networkx>=3.2
```

PennyLane 0.44.0 is current as of Jan 2026.

2) `requirements-photonic-legacy.txt` (optional photonic plugin track)

```
pennylane==0.29.0
pennylane-sf
strawberryfields==0.23.0
numpy<2.0
scipy<2.0
pyyaml>=6.0
```

- Install plugin via `pip install pennylane-sf`.
- Compatibility warning (0.29 max) is explicit in plugin docs.
- The SF “Fock device” is documented here (legacy track).

3) Example config: `examples/config_small.yaml`

This is intentionally small so the team can run it in minutes and then scale.

```yaml
problem:
  name: "manzanillo_small_demo"
  budget: 8.0
  penalty_lambda: 8.0
  include_pairwise_synergy: true
  synergy_scale: 0.2

candidates:
  # Each candidate is a decision variable x_{i,k} with a cost and a
precomputed utility
  # Utility here already represents aggregated expected avoided loss
contribution.
  - id: "bridge_1_seismic"
    cost: 2.0
    utility: 5.0
  - id: "substation_1_seismic"
    cost: 2.5
    utility: 6.0
  - id: "water_pump_1_flood"
    cost: 1.5
    utility: 4.0
  - id: "road_node_1_flood"
    cost: 1.2
    utility: 3.2
  - id: "port_gate_1_camera"
    cost: 1.8
    utility: 4.4
  - id: "hillside_1_fire_cam"
    cost: 2.2
    utility: 5.2

qaoa:
  layers: 2
  steps: 120
  lr: 0.08
  seed: 123
  device: "lightning.qubit"   # "default.qubit" also works
```

In the real project, `utility` should be computed from your hazard scenarios + asset values (expected avoided loss). This scaffold assumes you already have those coefficients from the risk layer pipeline.

4) Code: configuration model (`src/manzanillo_qc/config.py`)

```
from __future__ import annotations
from typing import List, Optional
from pydantic import BaseModel, Field


class Candidate(BaseModel):
    id: str
    cost: float = Field(gt=0)
    utility: float  # expected avoided loss contribution
(aggregated), >=0


class ProblemConfig(BaseModel):
    name: str = "manzanillo_sensor_placement"
    budget: float = Field(gt=0)
    penalty_lambda: float = Field(gt=0, default=10.0)
    include_pairwise_synergy: bool = True
    synergy_scale: float = 0.15


class QAOAConfig(BaseModel):
    layers: int = Field(ge=1, default=2)
    steps: int = Field(ge=1, default=100)
    lr: float = Field(gt=0, default=0.05)
    seed: int = 123
    device: str = "lightning.qubit"
    shots: Optional[int] = None  # None => analytic mode when
supported


class AppConfig(BaseModel):
    problem: ProblemConfig
    candidates: List[Candidate]
    qaoa: QAOAConfig
```

5) Build the QUBO (`src/manzanillo_qc/qubo.py`)

This encodes:

- maximize: **sum utility$_i$ x$_i$**
- subject to: $\mathbf{sum\ cost_i\ x_i \le B}$
- implemented as unconstrained QUBO using a penalty on budget violation.

We use a simple quadratic penalty:

$$\lambda \cdot \max(0, \sum c_i x_i - B)^2$$

For a starter, we approximate by penalizing $(\sum c_i x_i - B)^2$ directly (works well when you tune λ).

```
from __future__ import annotations
import numpy as np


def build_qubo_linear_plus_budget_penalty(
    utilities: np.ndarray,
    costs: np.ndarray,
    budget: float,
    lam: float,
    include_pairwise_synergy: bool = True,
    synergy_scale: float = 0.15,
) -> tuple[np.ndarray, float]:
    """
    Return (Q, const) so that the QUBO objective is:
        maximize  x^T Q x + const
    with x in {0,1}^n.

    We maximize:
        sum_i utilities[i] * x_i  - lam * (sum_i costs[i] * x_i -
budget)^2
    Optionally add mild pairwise synergy terms to encourage
redundancy/diversity.
    """

    n = len(utilities)
    Q = np.zeros((n, n), dtype=float)
    const = -lam * (budget**2)
```

```
    # Linear utility term contributes to diagonal in QUBO form
    for i in range(n):
        Q[i, i] += utilities[i]

    # Budget penalty expansion:
    # -lam * (c^T x - B)^2 = -lam * (x^T (c c^T) x - 2B c^T x + B^2)
    # => add -lam * (c c^T) to Q (quadratic)
    # => add +2 lam B c_i to diagonal (linear)
    for i in range(n):
        Q[i, i] += 2.0 * lam * budget * costs[i]
        for j in range(n):
            Q[i, j] += -lam * costs[i] * costs[j]

    # Optional simple synergy (toy): encourage selecting
complementary sensors.
    # In real use: synergy should be derived from coverage overlap
or redundancy.
    if include_pairwise_synergy:
        rng = np.random.default_rng(0)
        for i in range(n):
            for j in range(i + 1, n):
                # Small positive bonus for pairs (sparse random for
starter)
                if rng.random() < 0.15:
                    Q[i, j] += synergy_scale
                    Q[j, i] += synergy_scale

    return Q, const
```

6) Convert QUBO → Ising Hamiltonian (`src/manzanillo_qc/ising.py`)

PennyLane QAOA typically uses an **Ising cost Hamiltonian** with Pauli Z terms. This converts binary $x \in \{0,1\}$ into spins $z \in \{-1,+1\}$ using:

$$x = \frac{1-z}{2}$$

```
from __future__ import annotations
import numpy as np
import pennylane as qml


def qubo_to_ising(Q: np.ndarray, offset: float = 0.0):
    """
    Convert maximize x^T Q x + offset, x in {0,1}^n
    to minimize Ising H = sum_i h_i Z_i + sum_{i<j} J_ij Z_i Z_j +
const

    We return (H, const) where H is a PennyLane Hamiltonian.
    Note: QUBO is maximize; QAOA typically minimizes expectation, so
we negate.
    """
    Q = np.array(Q, dtype=float)
    n = Q.shape[0]

    # Symmetrize
    Q = 0.5 * (Q + Q.T)

    # Convert:
    # x_i = (1 - z_i)/2
    # x_i x_j = (1 - z_i - z_j + z_i z_j)/4
    # Build energy E(z) for - (maximize objective) => minimize
(-objective)
    h = np.zeros(n)
    J = {}

    const = 0.0
```

```
    # Diagonal terms: Q_ii x_i
    for i in range(n):
        const += -Q[i, i] * 0.5  # because x_i = (1 - z_i)/2
        h[i] += -Q[i, i] * (-0.5)  # coefficient on z_i

    # Off-diagonal terms: 2 * sum_{i<j} Q_ij x_i x_j (since
symmetric x^TQx includes both)
    for i in range(n):
        for j in range(i + 1, n):
            q = Q[i, j]
            if abs(q) < 1e-12:
                continue

            # term q * x_i x_j
            # contributes to const: q*(1)/4
            # z_i: q*(-1)/4, z_j: q*(-1)/4
            # z_i z_j: q*(1)/4
            const += -q * (1.0 / 4.0)
            h[i] += -q * (-1.0 / 4.0)
            h[j] += -q * (-1.0 / 4.0)
            J[(i, j)] = J.get((i, j), 0.0) + (-q * (1.0 / 4.0))

    # Add offset (maximize) becomes -offset in minimization
    const += -offset

    # Build PennyLane Hamiltonian
    coeffs = []
    ops = []

    for i in range(n):
        if abs(h[i]) > 1e-12:
            coeffs.append(h[i])
            ops.append(qml.PauliZ(i))

    for (i, j), val in J.items():
        if abs(val) > 1e-12:
            coeffs.append(val)
            ops.append(qml.PauliZ(i) @ qml.PauliZ(j))

    H = qml.Hamiltonian(coeffs, ops)
    return H, const
```

7) QAOA runner (`src/manzanillo_qc/qaoa.py`)

Uses PennyLane's QAOA building blocks (`qml.qaoa`)

```
from __future__ import annotations
import numpy as np
import pennylane as qml


def solve_with_qaoa(
    cost_h: qml.Hamiltonian,
    n_wires: int,
    layers: int = 2,
    steps: int = 100,
    lr: float = 0.05,
    device_name: str = "lightning.qubit",
    shots: int | None = None,
    seed: int = 123,
):
    dev = qml.device(device_name, wires=n_wires, shots=shots)

    mixer_h = qml.qaoa.x_mixer(wires=range(n_wires))

    # parameters: gammas and betas for p layers
    rng = np.random.default_rng(seed)
    params = np.array(
        [rng.normal(scale=0.1, size=layers), rng.normal(scale=0.1,
size=layers)],
        dtype=float,
    )

    @qml.qnode(dev, diff_method="best")
    def circuit(params):
        gammas, betas = params

        # Start in |+>^n
        for w in range(n_wires):
            qml.Hadamard(wires=w)

        # QAOA layers
```

```
        for l in range(layers):
            qml.qaoa.cost_layer(gammas[l], cost_h)
            qml.qaoa.mixer_layer(betas[l], mixer_h)

        return qml.expval(cost_h)

    opt = qml.AdamOptimizer(stepsize=lr)

    history = []
    for t in range(steps):
        params, val = opt.step_and_cost(circuit, params)
        history.append(float(val))

    return {
        "best_cost": min(history),
        "final_cost": history[-1],
        "history": history,
        "params": params,
    }


def sample_bitstrings(params, cost_h, n_wires, layers,
device_name="lightning.qubit", shots=2000):
    dev = qml.device(device_name, wires=n_wires, shots=shots)
    mixer_h = qml.qaoa.x_mixer(wires=range(n_wires))

    @qml.qnode(dev)
    def sampler(params):
        gammas, betas = params
        for w in range(n_wires):
            qml.Hadamard(wires=w)
        for l in range(layers):
            qml.qaoa.cost_layer(gammas[l], cost_h)
            qml.qaoa.mixer_layer(betas[l], mixer_h)

        return qml.sample(wires=range(n_wires))

    samples = sampler(params)
    # Convert Z-basis samples {0,1} to bitstrings
    # PennyLane returns 0/1 for measurement outcomes in
computational basis
```

```
    return samples
```

8) Glue: load config, build instance, run (`src/manzanillo_qc/cli.py`)

```
from __future__ import annotations
from pathlib import Path
import yaml
import numpy as np
from rich import print
import typer

from manzanillo_qc.config import AppConfig
from manzanillo_qc.qubo import build_qubo_linear_plus_budget_penalty
from manzanillo_qc.ising import qubo_to_ising
from manzanillo_qc.qaoa import solve_with_qaoa, sample_bitstrings

app = typer.Typer()


@app.command()
def run(config_path: str = typer.Argument(..., help="Path to YAML
config")):
    cfg = AppConfig(**yaml.safe_load(Path(config_path).read_text()))

    utilities = np.array([c.utility for c in cfg.candidates],
dtype=float)
    costs = np.array([c.cost for c in cfg.candidates], dtype=float)

    Q, const = build_qubo_linear_plus_budget_penalty(
        utilities=utilities,
        costs=costs,
        budget=cfg.problem.budget,
        lam=cfg.problem.penalty_lambda,

include_pairwise_synergy=cfg.problem.include_pairwise_synergy,
        synergy_scale=cfg.problem.synergy_scale,
    )

    cost_h, ising_const = qubo_to_ising(Q, offset=const)

    res = solve_with_qaoa(
        cost_h=cost_h,
```

```
        n_wires=len(cfg.candidates),
        layers=cfg.qaoa.layers,
        steps=cfg.qaoa.steps,
        lr=cfg.qaoa.lr,
        device_name=cfg.qaoa.device,
        shots=cfg.qaoa.shots,
        seed=cfg.qaoa.seed,
    )

    print("[bold]QAOA run complete[/bold]")
    print({"final_cost": res["final_cost"], "best_cost":
res["best_cost"]})

    # Sample and decode a candidate solution
    samples = sample_bitstrings(
        params=res["params"],
        cost_h=cost_h,
        n_wires=len(cfg.candidates),
        layers=cfg.qaoa.layers,
        device_name=cfg.qaoa.device,
        shots=2000,
    )

    # Take the most frequent bitstring
    bitstrings = ["".join(map(str, row.tolist())) for row in
samples]
    from collections import Counter
    top = Counter(bitstrings).most_common(1)[0][0]

    x = np.array(list(map(int, top)), dtype=int)
    total_cost = float(np.dot(costs, x))
    total_util = float(np.dot(utilities, x))

    selected = [cfg.candidates[i].id for i in range(len(x)) if x[i]
== 1]

    print("[bold]Top sampled solution[/bold]")
    print({"selected": selected, "total_cost": total_cost,
"linear_utility": total_util})
    if total_cost > cfg.problem.budget:
```

```
        print("[yellow]Warning: budget violated; increase
penalty_lambda or add slack-variable encoding.[/yellow]")


if __name__ == "__main__":
    app()
```

Run it:

```
python -m pip install -r requirements.txt
python -m manzanillo_qc.cli run examples/config_small.yaml
```

How to extend this (the “real Manzanillo” version)

Right now the config expects each candidate (i, k) has a single `utility`. In production, compute:

$$\text{utility}(i,k) \approx \sum_{h,s} p_{h,s} \sum_{a \in \mathcal{A}} L_{a,s}^{(h)}(0) \cdot r_{a,s}^{(h)}(i,k)$$

So you’ll add a module that:

- loads **asset classes** and values V_a,
- loads **scenario sets** Ω_h (from flood/wildfire/seismic pipelines),
- computes $r_{a,s}^{(h)}(i,k)$ from lead-time gains / uncertainty reduction / SOP effectiveness,
- outputs candidate utilities + costs to YAML/JSON for optimization.

That keeps the quantum piece clean and reusable.

Photonic note (important)

If you want “photonic in the loop” *inside* PennyLane, you must use the **legacy** `pennylane-sf` plugin, which is only compatible up to PennyLane 0.29.
That’s why it is recommended:

- use **modern PennyLane** for QAOA optimization development (Track A),
- and run photonic circuit experiments either in a **separate legacy env** or directly in Strawberry Fields for photonics-specific R&D.

Checklist

- ✅ Start with `examples/config_small.yaml` and confirm QAOA runs end-to-end.
- ✅ Replace random “synergy” with a real overlap/redundancy model (pairwise terms).
- ✅ Implement a “utility builder” from your hazard scenario pipeline.
- ✅ Add benchmarks: greedy + MILP (so you can prove quality/time tradeoffs).
- ✅ Keep photonic experiments in a separate env until you decide whether to rely on the legacy plugin.